Package 'CrossingTools'

November 12, 2025

Title Tools for Cross Optimization in Plant Breeding Programs

Version 1.0.1

Description Mate allocation is a critical component of plant breeding programs, aiming to generate superior progenies and improved populations over time. While numerous methodologies for mate allocation exist, few software tools offer a unified and scalable framework that integrates these methodologies in a user-friendly manner. 'CrossingTools' addresses this gap by providing a flexible, efficient solution for optimizing mating decisions across a wide range of breeding schemes. The package supports genomic BLUP and selection indices for various variance structures and implements multiple selection criteria to evaluate cross potential, including expected mean, optimal haploid value (OHV), and superior progeny value (SPV). Different methods are provided for calculating additive and dominance segregation variances for both inbred and heterozygous parents, making 'CrossingTools' suitable for a wide range of breeding strategies. After criterion calculation, users can rank potential crosses or apply the built-in optimal crossing selection routine, which balances genetic gain and diversity to support long-term improvement. The package combines R's scripting flexibility with high-performance C++ routines via 'Rcpp' and 'RcppArmadillo', ensuring scalability for large populations with dense marker data.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE, roclets = c(` `collate", ` `namespace", ` `rd"))

ByteCompile true

RoxygenNote 7.3.3

Depends R (>= 4.0.0)

Imports ggplot2,
ggridges,
Rcpp,
RcppParallel

LinkingTo Rcpp,
RcppArmadillo,
RcppParallel

SystemRequirements C++17

Suggests knitr,
rmarkdown,
testthat (>= 3.0.0)

Config/testthat/edition 3

VignetteBuilder knitr

URL https://github.com/svenernstW/CrossingTools

BugReports https://github.com/svenernstW/CrossingTools/issues

R topics documented:

`calculate_desired_gains` *Calculation of desired gains index (Werner et al.)*

Description

Calculates the desired gains index based on multi-trait BLUPs and a chosen (co)variance matrix option.

Usage

```
calculate_desired_gains(
  A,
  V,
  V.approx,
  gains = NULL,
  use.V = TRUE,
  use.marginal.V = FALSE,
  use.V.approx = FALSE,
  n.Threads = 4L
)
```

Arguments

A	A matrix (n_genotype x n_trait) of multi-trait BLUPs.
V	Numeric matrix ((n_genotype*n_trait) x (n_genotype*n_trait)) with the posterior variance var($\tilde{g}$). Required if use.V or use.marginal.V is TRUE.
V.approx	Numeric matrix (n_trait x n_trait) giving an approximation of var($\tilde{g}$), e.g. cov(A). Required if use.V.approx is TRUE.
gains	Numeric vector (length = n_trait) of desired gains. If NULL, uses equal weights (rep(1, n_trait)).
use.V	Logical. If TRUE, uses each genotype's own trait-block V_g to compute a per-genotype index (i.e., $w_g = V_g^{-1} d$).
use.marginal.V	Logical. If TRUE, uses the *marginal* posterior variance (mean of per-genotype trait-blocks from V).
use.V.approx	Logical. If TRUE, uses V.approx.
n.Threads	Number of OpenMP threads.

Value

A one-column data.frame with the desired-gains index for each genotype.

---

calculate_optimal_haploid_value

Optimal Haplotypic Value (OHV) for proposed crosses

Description

Computes the OHV for each cross using block-wise local GEBVs. If haplotype.blocks is not supplied (or empty), each marker column of M is treated as a separate block.

Usage

```
calculate_optimal_haploid_value(
  crosses,
  haplotype.blocks = NULL,
  M,
  U,
  n.Threads = 4L
)
```

Arguments

crosses	A numeric matrix or data.frame with two columns (P1, P2), giving indices of parents (rows of M) for each proposed cross.
haplotype.blocks	A list of integer vectors. Each vector contains marker indices (columns of M) belonging to that block. If NULL or length 0, each marker is used as a separate block.
M	Numeric genotype/marker matrix (individuals x markers).
U	Numeric vector of marker effects (length ncol(M)).
n.Threads	Integer larger 1. OpenMP threads (if enabled at compile time).

Value

A data.frame with columns `OHV`.

`calculate_variances_4W_DH` *Segregation variance for DH lines from specified four- or three-way crosses (Allier)*

Description

Calculate expected genomic estimated breeding values (EGBV), segregation variance, and superior progeny value (SPV) for doubled haploid (DH) lines derived after `t` rounds of random mating.

Usage

```
calculate_variances_4W_DH(
  crosses,
  genetic.map,
  M,
  U,
  t,
  intensity,
  covariance = FALSE,
  calculate.gains = FALSE,
  gains = NULL,
  n.Threads = 4L
)
```

Arguments

`crosses`	A data.frame or matrix (n_crosses x 4) of parental indices (row indices into `M`) specifying four- or three-way crosses. For three-way crosses, the first two crossing partners are the same. You can also compute the variance for two heterozygous individuals by passing haplotypes in `M`; then indices refer to haplotypes.
`genetic.map`	A data.frame with columns: • `site`: integer marker index (1..ncol(M)) • `chr`: chromosome identifier • `pos`: position on the chromosome in morgan All markers in `M` must appear in `genetic.map$site`.
`M`	Numeric marker matrix (genotypes/haplotypes in rows, markers in columns) with entries in `c(0, 2)` corresponding to `c("AA", "BB")`. For heterozygotes, store haplotypes in `M` (each individual uses two rows).
`U`	Numeric matrix of marker effects (markers in rows, traits in columns). Must have `nrow(U) == ncol(M)`.
`t`	Integer. Number of random-mating generations before DH creation.
`intensity`	Numeric. Standardized selection differential.
`covariance`	Logical. If `TRUE`, also compute the segregation covariance matrix between all supplied traits.

`calculate.gains`
: Logical. If `TRUE`, calculate the desired gains index based on segregation (co)variances. This only works if covariance is TRUE, else will be ignored.

`gains`
: a vector of length equal to the number of traits with values representing the desired gains

`n.Threads`
: Integer (default 4). Number of OpenMP threads (if enabled at compile time).

Value

If `covariance = TRUE`, a list with element `cross_values` (a data.frame) whose columns are named `EGBV1, var1, SPV1, EGBV2, ...` and a list with element `covariances` with segregation covariance matrix for each cross. If `covariance = TRUE`, a list with element `cross_values` (a data.frame) whose columns are named `EGBV1, var1, SPV1, EGBV2, ...`, element `gains` (a data.frame) with `EGBV_DG, varDG, SPVDG` for the desired gains index and a list with element `covariances` with segregation covariance matrix for each cross. If `covariance = FALSE`, a data.frame with the same columns `EGBV1, var1, SPV1, EGBV2, ....`

Examples

```
## Not run:
out <- calculate_variances_DH(
  crosses = matrix(c(1,2, 3,4), ncol = 4, byrow = TRUE),
  genetic.map = data.frame(site = 1:ncol(M), chr = 1, pos = seq_len(ncol(M))),
  M = M,
  U = matrix(rnorm(ncol(M)), ncol = 1),
  t = 0,
  intensity = 1.0,
  covariance = FALSE
)

## End(Not run)
```

---

calculate_variances_4W_RIL

Segregation variance for RIL from specified four- or three-way crosses (Allier)

Description

Calculate expected genomic estimated breeding values (EGBV), segregation variance, and superior progeny value (SPV) for recombinant inbred lines(RIL) derived after `t` rounds of random mating.

Usage

```
calculate_variances_4W_RIL(
  crosses,
  genetic.map,
  M,
  U,
  t,
  intensity,
  covariance = FALSE,
```

```
  calculate.gains = FALSE,
  gains = NULL,
  n.Threads = 4L
)
```

Arguments

crosses	A data.frame or matrix (n_crosses x 4) of parental indices (row indices into M) specifying four- or three-way crosses. For three-way crosses, the first two crossing partners are the same. You can also compute the variance for two heterozygous individuals by passing haplotypes in M; then indices refer to haplotypes.
genetic.map	A data.frame with columns: • site: integer marker index (1..ncol(M)) • chr: chromosome identifier • pos: position on the chromosome in morgan All markers in M must appear in genetic.map$site.
M	Numeric marker matrix (genotypes/haplotypes in rows, markers in columns) with entries in c(0, 2) corresponding to c("AA", "BB"). For heterozygotes, store haplotypes in M (each individual uses two rows).
U	Numeric matrix of marker effects (markers in rows, traits in columns). Must have nrow(U) == ncol(M).
t	Integer. Number of random-mating generations before RIL creation.
intensity	Numeric. Standardized selection differential.
covariance	Logical. If TRUE, also compute the segregation covariance matrix between all supplied traits.
calculate.gains	Logical. If TRUE, calculate the desired gains index based on segregation (co)variances. This only works if covariance is TRUE, else will be ignored.
gains	a vector of length equal to the number of traits with values representing the desired gains
n.Threads	Integer (default 4). Number of OpenMP threads (if enabled at compile time).

Value

If covariance = TRUE, a list with element cross_values (a data.frame) whose columns are named EGBV1, var1, SPV1, EGBV2, ... and a list with element covariances with segregation covariance matrix for each cross. If covariance = TRUE, a list with element cross_values (a data.frame) whose columns are named EGBV1, var1, SPV1, EGBV2, ..., element gains (a data.frame) with EGBV_DG, varDG, SPVDG for the desired gains index and a list with element covariances with segregation covariance matrix for each cross. If covariance = FALSE, a data.frame with the same columns EGBV1, var1, SPV1, EGBV2, ...

Examples

```
## Not run:
out <- calculate_variances_RIL(
  crosses = matrix(c(1,2, 3,4), ncol = 4, byrow = TRUE),
  genetic.map = data.frame(site = 1:ncol(M), chr = 1, pos = seq_len(ncol(M))),
  M = M,
  U = matrix(rnorm(ncol(M)), ncol = 1),
```

```
  t = 0,
  intensity = 1.0,
  covariance = FALSE
)

## End(Not run)
```

`calculate_variances_DH` *Segregation variance for DH lines from specified crosses*

Description

Calculate expected genomic estimated breeding values (EGBV), segregation variance, and superior progeny value (SPV) for doubled haploid (DH) lines derived after `t` rounds of random mating.

Usage

```
calculate_variances_DH(
  crosses,
  genetic.map,
  M,
  U,
  t,
  intensity,
  covariance = FALSE,
  calculate.gains = FALSE,
  gains = NULL,
  method = "osthushenrich",
  n.Threads = 4L
)
```

Arguments

`crosses`	A data.frame or matrix (n_crosses x 2) of parental indices (row indices into `M`) specifying the crosses.
`genetic.map`	A data.frame with columns: • `site`: integer marker index (1..ncol(M)) • `chr`: chromosome identifier (numeric) • `pos`: position on the chromosome in morgan All markers in `M` must appear in `genetic.map$site`.
`M`	Numeric marker matrix (markers in columns, genotypes in rows) with entries in `c(0, 2)` corresponding to `c("AA", "BB")`.
`U`	Numeric matrix of marker effects (markers in rows, traits in columns). Must have `nrow(U) == ncol(M)`.
`t`	Integer. Number of random-mating generations before DH creation.
`intensity`	Double. Standardized selection differential.
`covariance`	Logical. If `TRUE`, also compute the segregation covariance matrix between all supplied traits.

calculate.gains
: Logical. If TRUE, calculate the desired gains index based on segregation (co)variances. This only works if covariance is TRUE, else will be ignored.

gains
: a vector of length equal to the number of traits with values representing the desired gains

method
: Character. Which method to use; one of c("osthushenrich", "lehermeier").

n.Threads
: Integer (default 4). Number of OpenMP threads (if enabled at compile time).

Value

If covariance = TRUE, a list with element cross_values (a data.frame) whose columns are named EGBV1, var1, SPV1, EGBV2, ... and a list with element covariances with segregation covariance matrix for each cross. If covariance = TRUE, a list with element cross_values (a data.frame) whose columns are named EGBV1, var1, SPV1, EGBV2, ..., element gains (a data.frame) with EGBV_DG, varDG, SPVDG for the desired gains index and a list with element covariances with segregation covariance matrix for each cross. If covariance = FALSE, a data.frame with the same columns EGBV1, var1, SPV1, EGBV2, ...

Examples

```
## Not run:
out <- calculate_variances_DH(
  crosses = matrix(c(1,2, 3,4), ncol = 2, byrow = TRUE),
  genetic.map = data.frame(site = 1:ncol(M), chr = 1, pos = seq_len(ncol(M))),
  M = M,
  U = matrix(rnorm(ncol(M)), ncol = 1),
  t = 0,
  intensity = 1.0,
  covariance = FALSE,
  method = "lehermeier"
)

## End(Not run)
```

---

calculate_variances_F1

Additive and dominance segregation variance of families from specified crosses (Wolfe)

Description

Calculate expected genomic estimated breeding value (EGBV), expected total genetic value (ETGV), additive and dominance segregation variance, superior progeny value with additive variance (SPV), and superior progeny value including dominance variance (TSPV) for F1 families.

Usage

```
calculate_variances_F1(
  crosses,
  genetic.map,
  hap1,
  hap2,
```

```
  U,
  D,
  intensity,
  covariance,
  calculate.gains = FALSE,
  gains = NULL,
  n.Threads = 4L
)
```

Arguments

crosses	A data.frame or matrix (n_crosses x 2) of parental indices (row indices into hap1/hap2) specifying the crosses.
genetic.map	A data.frame with columns: • site: integer marker index (1..ncol(hap1)) • chr: chromosome identifier (numeric or factor) • pos: position on the chromosome in morgan All markers in hap1/hap2 must appear in genetic.map$site.
hap1	Numeric haplotype matrix (individuals x markers) for the first haplotype, entries in c(0, 1) for c("A", "B").
hap2	Numeric haplotype matrix (individuals x markers) for the second haplotype, entries in c(0, 1) for c("A", "B").
U	Numeric matrix of additive marker effects (markers in rows, traits in columns). Must have nrow(U) == ncol(hap1).
D	Numeric matrix of dominance marker effects (markers in rows, traits in columns). Must have nrow(D) == ncol(hap1) and ncol(D) == ncol(U).
intensity	Double. Standardized selection differential (used for SPV).
covariance	Logical. If TRUE, also compute the segregation covariance matrix between all supplied traits.
calculate.gains	Logical. If TRUE, calculate the desired gains index based on segregation (co)variances. This only works if covariance is TRUE, else will be ignored.
gains	a vector of length equal to the number of traits with values representing the desired gains
n.Threads	Integer (default 4). Number of OpenMP threads (if enabled at compile time).

Value

If covariance = TRUE, a list with element cross_values (a data.frame) whose columns are named EGBV1, ETGV1, varA1, SPV1, varD, SPTV1, EGBV2, ... and a list with element covariances with segregation covariance matrix for each cross. If covariance = TRUE, a list with element cross_values (a data.frame) whose columns are named EGBV1, ETGV1, varA1, SPV1, varD, SPTV1, EGBV2, ..., element gains (a data.frame) with EGBVDG, ETGVDG, varADG, SPVDG, varDDG, SPTVDG for the desired gains index and a list with element covariances with segregation covariance matrix for each cross. If covariance = FALSE, a data.frame with the same columns EGBV1, ETGV1, varA1, SPV1, varD, SPTV1, EGBV2, ...

Examples

```
## Not run:
# toy shapes only
crosses <- matrix(c(1,2, 3,4), ncol = 2, byrow = TRUE)
hap1 <- matrix(sample(0:1, 20, TRUE), nrow = 5)  # 5 inds x 4 markers
hap2 <- matrix(sample(0:1, 20, TRUE), nrow = 5)
genetic.map <- data.frame(site = 1:ncol(hap1), chr = c(1,1,2,2), pos = c(0,10,0,10))
U <- matrix(rnorm(ncol(hap1)), ncol = 1)
D <- matrix(rnorm(ncol(hap1)), ncol = 1)
out <- calculate_variances_F1(crosses, genetic.map, hap1, hap2, U, D, intensity = 1.0, covariance = FALSE)

## End(Not run)
```

`calculate_variances_RIL` *Segregation variance for recombinant inbred lines from specified crosses*

Description

Calculate expected genomic estimated breeding values (EGBV), segregation variance, and superior progeny value (SPV) for recombinant inbred lines (RIL) derived after `t` rounds of random mating.

Usage

```
calculate_variances_RIL(
  crosses,
  genetic.map,
  M,
  U,
  t,
  intensity,
  covariance = FALSE,
  calculate.gains = FALSE,
  gains = NULL,
  method = "osthushenrich",
  n.Threads = 4L
)
```

Arguments

`crosses`	A data.frame or matrix (n_crosses x 2) of parental indices (row indices into `M`) specifying the crosses.
`genetic.map`	A data.frame with columns: • `site`: integer marker index (1..ncol(M)) • `chr`: chromosome identifier (numeric) • `pos`: position on the chromosome in morgan All markers in `M` must appear in `genetic.map$site`.
`M`	Numeric marker matrix (markers in columns, genotypes in rows) with entries in `c(0, 2)` corresponding to `c("AA", "BB")`.

U	Numeric matrix of marker effects (markers in rows, traits in columns). Must have nrow(U) == ncol(M).
t	Integer. Number of random-mating generations before RIL creation.
intensity	Double. Standardized selection differential.
covariance	Logical. If TRUE, also compute the segregation covariance matrix between all supplied traits.
calculate.gains	Logical. If TRUE, calculate the desired gains index based on segregation (co)variances. This only works if covariance is TRUE, else will be ignored.
gains	a vector of length equal to the number of traits with values representing the desired gains
method	Character. Which method to use; one of c("osthushenrich", "lehermeier").
n.Threads	Integer (default 4). Number of OpenMP threads (if enabled at compile time).

Value

If covariance = TRUE, a list with element cross_values (a data.frame) whose columns are named EGBV1, var1, SPV1, EGBV2, ... and a list with element covariances with segregation covariance matrix for each cross. If covariance = TRUE, a list with element cross_values (a data.frame) whose columns are named EGBV1, var1, SPV1, EGBV2, ..., element gains (a data.frame) with EGBV_DG, varDG, SPVDG for the desired gains index and a list with element covariances with segregation covariance matrix for each cross. If covariance = FALSE, a data.frame with the same columns EGBV1, var1, SPV1, EGBV2, ...

Examples

```
## Not run:
out <- calculate_variances_RIL(
  crosses = matrix(c(1,2, 3,4), ncol = 2, byrow = TRUE),
  genetic.map = data.frame(site = 1:ncol(M), chr = 1, pos = seq_len(ncol(M))),
  M = M,
  U = matrix(rnorm(ncol(M)), ncol = 1),
  t = 0,
  intensity = 1.0,
  covariance = FALSE,
  method = "lehermeier"
)

## End(Not run)
```

---

create_cross_plan *Creation of a plan of all potential crosses for CrossingTools*

Description

Creation of a plan of all potential crosses for CrossingTools

Usage

```
create_cross_plan(
  candidates = NULL,
  male_candidates = NULL,
  female_candidates = NULL
)
```

Arguments

candidates	A vector of integers that identify each genotype. Should be consistent with genotypic data. Use this function if there are no sexes.
male_candidates	A vector of integers that identify each male genotype. Ignored if candidates is supplied. Should be consistent with genotypic data.
female_candidates	A vector of integers that identify each female genotype. Ignored if candidates is supplied. Should be consistent with genotypic data.

Value

A two-column data.frame with the the combinations of all supplied individuals. If male and female candidates are supplied, the first column corresponds to males, and the second to females

---

evaluate_cross_plan *Evaluate a cross plan by averaging value metrics across all crosses*

Description

Evaluate a cross plan by averaging value metrics across all crosses

Usage

```
evaluate_cross_plan(cross_plan, cross_param)
```

Arguments

cross_plan	A data.frame with two columns as produced by `create_cross_plan()`.
cross_param	Either a data.frame (when covariance = FALSE) or a list with element `$cross_values` (data.frame) when covariance = TRUE, produced by calculate_*.

Value

A single-row data.frame containing the mean across all crosses for each available value metric per trait (among EGBV, ETGV, SPV, SPTV, TSPV), with columns named `mean_<METRIC><trait_index>`, plus `n_crosses`.

`optimal_cross_selection` *Optimal cross selection via a genetic algorithm (with optional fixed crosses)*

Description

Uses a simple genetic algorithm (GA) to choose `ncrosses` parent pairs that balance an average criterion (utility `u`) and genomic similarity according to a target trade-off angle. Fixed crosses (if any) are forced into the final solution, and the GA fills the remaining slots from `crosses`.

Usage

```
optimal_cross_selection(
  crosses,
  fixed.crosses = NULL,
  ncrosses,
  target.angle,
  u,
  u.fixed = NULL,
  G,
  propability.mutate = 0.01,
  n.mutate = 2,
  n.select = 500,
  n.pop = 10000,
  max.generation = 100,
  max.iteration = 100,
  angle.penalty = 0.5,
  n.Threads = 4L
)
```

Arguments

`crosses`	integer matrix (K x 2). Candidate *variable* crosses (1-based indices referring to rows/columns of `G`). Must have exactly two columns (P1, P2); selfings are not allowed.
`fixed.crosses`	integer matrix (F x 2) or `NULL`. Crosses that must be included in the final plan. Can have 0 rows. Must use the same indexing as `crosses`. Selfings are not allowed.
`ncrosses`	integer. Total number of crosses in the final plan (variable + fixed). Must be $\geq$ number of rows in `fixed.crosses`.
`target.angle`	numeric (radians). Target trade-off angle between the average criterion and the similarity objective. Smaller angles emphasize utility `u`; larger angles emphasize similarity control.
`u`	numeric vector (length = nrow(crosses)). Utility (average criterion) for each candidate cross in `crosses`.
`u.fixed`	numeric vector (length = nrow(fixed.crosses)) or `NULL`. Utility for each fixed cross; if `fixed.crosses` has 0 rows, this can be length-0.

G	numeric square matrix (nInd x nInd). Genomic relationship matrix used to evaluate similarity/diversity among parents; indices in crosses/fixed.crosses must lie in 1..nrow(G).
propability.mutate	numeric in [0,1]. Probability that a candidate solution mutates in the GA.
n.mutate	integer ≥ 0. Number of point mutations to apply when a mutation occurs.
n.select	integer ≥ 2. Number of candidate solutions selected per generation (selection pool size).
n.pop	integer ≥ 2. Population size (number of candidate solutions) per generation.
max.generation	integer ≥ 1. Maximum number of GA generations.
max.iteration	integer ≥ 1. Maximum number of iterations without improvement (early-stopping style).
angle.penalty	numeric ≥ 0. Penalty applied to solutions that deviate from target.angle (encourages the desired trade-off).
n.Threads	integer ≥ 1. Number of OpenMP threads to use.

Value

A list with elements :

crossPlan Integer matrix (ncrosses x 3): the selected *final* cross plan (includes fixed crosses) and an additional column i with the indeces in the original crosses data.frame.

uMax Numeric scalar: maximum attainable average criterion when optimizing only u.

uMin Numeric scalar: minimum attainable average criterion when optimizing only similarity.

simMax Numeric scalar: similarity corresponding to uMax.

simMin Numeric scalar: similarity corresponding to uMin.

uBest Numeric scalar: average criterion achieved by the *optimized* cross plan.

simBest Numeric scalar: similarity of the optimized cross plan.

angleBest Numeric scalar (radians): angle of the optimized solution relative to the target trade-off.

lenBest Numeric scalar: length (magnitude) of the optimized solution vector in the objective space.

---

plot_cross_plan *Plot cross plan ridge distributions per trait*

Description

Simulates per-cross distributions using EGBV as mean and the sum of all non-dominance variance columns (i.e., all var* except varD) as the variance. Draws ridgeline densities, overlays EGBV (blue circles) and SPV (red diamonds, if present), and adds a dotted vertical line at the per-trait average EGBV. One panel per trait (3 columns), each with its own x-axis scale.

Usage

```
plot_cross_plan(cross_plan, cross_param, traits = NULL)
```

Arguments

cross_plan	data.frame: two columns from create_cross_plan()
cross_param	data.frame or list with $cross_values from calculate_variances*()
traits	integer vector of trait indices to plot, or NULL for all

Value

ggplot object

---

u_from_g *Backsolve marker effects (mu) from individual effects (g)*

Description

Computes marker effects $\mu = (scalingFactor * M' * G^{-1}) * g$.

Usage

```
u_from_g(M, G, g, scaling.factor, tol = 1e-10, n.Threads = 4L)
```

Arguments

M	Numeric matrix (n x p): marker/genotype matrix (individuals in rows, markers in columns).
G	Numeric matrix (n x n): relationship matrix among individuals.
g	Numeric (n x k) or length-n vector: individual effects (one or more traits).
scaling.factor	Numeric scalar used to scale the GRM.
tol	Numeric scalar tolerance for near-singularity checks in G. Default: 1e-10.
n.Threads	Integer >= 1; OpenMP threads (if enabled at compile time). Default: 4L.

Value

A numeric p x k matrix of marker effects (mu_matrix).

Index